Informe de Laboratorio 05

Tema: Django-PARTE I

Nota

Estudiante	Escuela	Asignatura
Francis Willian Ayamamani Añasco fayamamani@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Programación Web 2 Semestre: I Código: GUIA-PRLD-001

Laboratorio	Tema	Duración
05	Django-PARTE I	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2023 - A	Del 30 Junio 2023	Al 30 Junio 2023

1. Tarea

- Creacion del sitio respecto al administrador:
  1. Creacion del models.py
  2. Creacion de las funciones de views.py
  3. Creacion de Templates
  4. Vincular las rutas en urls.py

2. MARCO CONCEPTUAL

- Editor Vim
  - Vim es un editor de texto muy eficiente para programar en consola que no tiene nada que envidiar a los editores gráficos.
- Instalar Vim en
  - GNU/Linux

```
apt-get install vim
```

  - MS Windows
    - Para descarga en sistemas MS Windows `https://www.vim.org/download.php`

- MacOS

```
brew install macvim
```

- Git
  - Git es un sistema de control de versiones creado por Linus Torvalds. Es eficiente, confiable.
- Instalar Git
  - GNU/Linux

```
apt-get install git
```

  - MS Windows
    - Para descarga en sistemas MS Windows `https://git-scm.com/download/win`
  - MacOS
    - Para descarga en sistemas MacOS `https://git-scm.com/download/mac`

```
brew install git
```

- python
  - Para un instalación local en sistemas operativos MS Windows utilice: https://www.python.org/downloads/windo
  - Tambien ud puede probar código en herramientas online: `https://www.programiz.com/python-programming/online-compiler/`
  - Python en Debian GNU/Linux:

Listing 1: Debian Linux viene con Python 3 y Python 2 preinstalados.

```
python3 --version
   Python 3.9.2
```

Listing 2: Instalemos pip, una herramienta que instalará y administrará los paquetes de programación que queramos usar en nuestros proyectos de desarrollo.

```
sudo apt-get install -y python3-pip
```

Listing 3: Paquetes y herramientas de desarrollo más para instalar para garantizar que tengamos una configuración sólida para nuestro entorno de programación:

```
sudo apt-get install build-essential libssl-dev libffi-dev
    python3-dev
```

  - Configurar un entorno virtual:
    - Los entornos virtuales permiten tener un espacio aislado en los proyectos Python
    - Garantizando que cada proyecto pueda tener su propio conjunto de dependencias que no interrumpirán a otros proyectos.
    - Manejando diferentes versiones de los paquetes. Esto es especialmente importante cuando se trabaja con paquetes de terceros.

- Puede varios entornos de programación.
- Cada entorno es un directorio en la que se ubicaran sus scripts.
- Usaremos el módulo venv , que es parte de la biblioteca estándar de Python.

Listing 4: Entorno de Linux-Instalemos venv escribiendo:

```
sudo apt install -y python3-venv
```

Listing 5: Entorno de Linux-Para crear un ambiente elija en qué directorio.

```
mkdir -p ~/unsa/pw2/labs/lab04/entorno
    cd ~/unsa/pw2/labs/lab04/entorno
```

Listing 6: Entorno de Linux-En el directorio crea un entorno virtual ejecutando el siguiente comando:

```
virtualenv -p python3 .
```

Listing 7: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
source ../bin/activate
```

3. URL de Repositorio Github

- URL del Repositorio GitHub para clonar o recuperar.
- `https://github.com/rescobedoq/pw2/tree/main/labs/lab05`
- URL para el laboratorio 05 en el Repositorio GitHub.
- `https://github.com/rescobedoq/pw2/tree/main/labs/lab05`

4. Resolucion de la tarea

4.1. Inicio

- Creacion de la carpeta de Trabajo

Listing 8: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
mkdir PWB
```

- Activar el entorno virtual

Listing 9: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
source {{Direccion de tu entorno Virtual}}
```

- Gestionar las versiones instaladas

Listing 10: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip list
```

- Instalar django

Listing 11: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip install Django
```

- Verificas

Listing 12: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
pip list
 //resultado:
    Package   Version
    ---------- -------
    asgiref   3.5.2
    Django    4.0.5 //Variable
    pip       22.0.4
    setuptools 62.1.0
    sqlparse  0.4.2
    wheel     0.37.1
```

- Crear el projecto

Listing 13: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
django-admin startproject {{nombre_del_proyecto}}
```

- RESULTADO

```
PWB/
|--- manage.py
|--- PWB/
    |   |--- __init__.py
    |   |--- asgi.py
    |   |--- settings.py
    |   |--- urls.py
```

- Crear el APP para el funcionamiento de una parte del projecto

Listing 14: Entorno de Linux-Ejecute el script de activación para activar el entorno virtual:

```
python manage.py startapp {{nombre_app}}
```

- RESULTADO

```
PWB/
|--- manage.py
#nombre de mi APP
|--- Decoraciones/
    |   |--- migration
    |   |--- __init__.py
    |   |--- admin.py
    |   |--- apps.py
    |   |--- models.py
```

```
|   |--- tests.py
|   |--- views.py
|   |--- urls.py
|--- PWB/
|   |--- __init__.py
|   |--- asgi.py
|   |--- settings.py
|   |--- urls.py
```

4.2. Explicacion del Codigo

Listing 15: Decoraciones/models.py

```python
from django.db import models

class Tipo(models.Model):
    Tip = models.CharField(max_length=50, default=None, null=True)

    def __str__(self):
        return self.Tip

class Producto(models.Model):
    nombre = models.CharField(max_length=25)
    descripcion = models.TextField()
    precio = models.CharField(max_length=25)

    class Meta:
        abstract = True

class catalogo(Producto):
    imagen = models.ImageField(null=True, blank=True, upload_to="img/")
    tipo = models.ForeignKey(Tipo, on_delete=models.CASCADE, null=True)

    def __str__(self):
        return self.nombre

class Carrito(Producto):
    adquirido = models.BooleanField()
    rutaI = models.CharField(null=True, max_length=50000)

class Pedido(models.Model):
    NombreC = models.CharField(null=True, max_length=50)
    ApellidosC = models.CharField(null=True, max_length=50)
    telefono = models.IntegerField(null=True)
    Correo = models.EmailField(max_length=254, default='')
    direccion = models.CharField(max_length=50)
    fecha = models.DateField()
    Carrito = models.ForeignKey(Carrito, on_delete=models.CASCADE, null=True)
```

- **Tipo**: Es un modelo que representa el tipo de producto. Tiene un campo Tip que es un CharField de máximo 50 caracteres de la cadena de texto. La función**str** devuelve la vista del admin, este caso sería el valor del campo Tip.
- **Producto**: Es una clase abstracta que representa un producto genérico. Tiene tres campos:

nombre, descripcion y precio. Estos campos son un CharField, un TextField y otro CharField respectivamente. Al declarar **abstract = True** en la clase Meta, se indica que este modelo es abstracto.

- **catalogo**: Este hereda de Producto y representa un producto en particular.Integra un campo imagen que es un ImageField para almacenar la imagen del producto, y un campo tipo que es una clave externa (**ForeignKey**) que se relaciona con el modelo Tipo.
- **Carrito**: Es un modelo que también hereda de Producto y representa un producto agregado al carrito de compras. Además de los campos heredados de Producto, tiene un campo adquirido que es un BooleanField para indicar si el producto ha sido adquirido, y un campo rutaI que es un CharField para almacenar la ruta de la imagen del producto.
- **Pedido**: Es un modelo que representa un pedido realizado por un cliente. Tiene varios campos como NombreC, ApellidosC, telefono, Correo, direccion, fecha y Carrito. Estos campos representan información relacionada con el pedido, como el nombre del cliente, apellidos, teléfono, correo, dirección, fecha del pedido, y un campo Carrito que es una clave externa (ForeignKey) que se relaciona con el modelo Carrito.

Listing 16: Decoraciones/models.py

```
from django.contrib import admin
from Decoraciones.models import Pedido,catalogo,Carrito,Tipo
# Register your models here.
admin.site.register(Tipo)
admin.site.register(Pedido)
admin.site.register(catalogo)
admin.site.register(Carrito)
```

- **La línea from django.contrib import admin** importa el módulo admin de Django, que proporciona las funcionalidades relacionadas con el panel de administración.
- Luego se importan los modelos Pedido, catalogo, Carrito y Tipo desde el módulo models de la aplicación Decoraciones.
- La función **admin.site.register(Modelo)** se utiliza para registrar cada modelo en el panel de administración. Aquí, se registran los modelos Tipo, Pedido, catalogo y Carrito.

Listing 17: Decoraciones/views.py

```
from django.shortcuts import render
from Decoraciones.models import catalogo
# Lista de elementos
list_producto=catalogo.objects.all()
#Pagina de Inicio
def Home(request):
    return render(request,"home.html",{"producs":list_producto})
def Bebidas(request):
    return render(request,"Bbs.html",{"producs":list_producto})
def Bocaditos(request):
    return render(request,"Bts.html",{"producs":list_producto})
def Equipos(request):
    return render(request,"Eps.html",{"producs":list_producto})
def Decoraciones(request):
    return render(request,"Dec.html",{"producs":list_producto})
```

- **from Decoraciones.models import catalogo**.Importa el modelo catalogo desde el archivo models.py del directorio Decoraciones. Esto permite acceder a los objetos de la clase catalogo definida en ese modelo.

- $\textbf{list}_producto = catalogo.objects.all(), obtienetodoslosobjetosdelaclasecatalogoalmacenadosylosasignaalavariable\textbf{list}_prod$

Listing 18: Decoraciones/urls.py

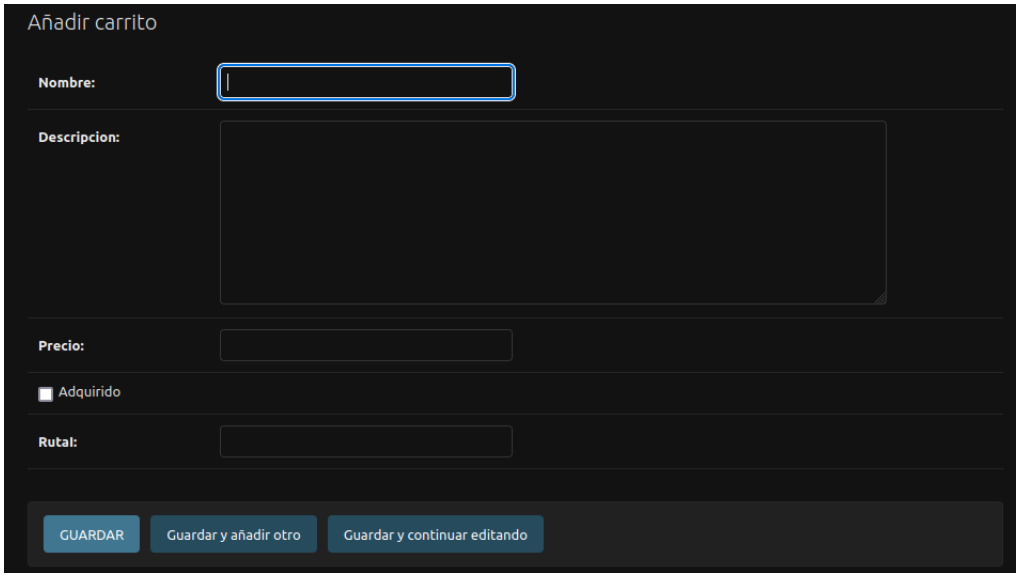
```
from django.urls import path
from Decoraciones import views
urlpatterns = [
    path('', views.Home, name="Inicio"),
    path('Bebidas/', views.Bebidas, name="Bebidas"),
    path('Bocaditos/', views.Bocaditos, name="Bocaditos"),
    path('Equipos/', views.Equipos, name="Equipos"),
    path('Decoraciones/', views.Decoraciones, name="Decoraciones"),
]
```

- **urlpatterns = [...]**, es una lista que contiene todas las rutas URL de la aplicación y las funciones de vista asociadas a cada una.
- **path(", views.Home, name=Ïnicio")**, define una ruta URL vacía (") que corresponde a la página de inicio de la aplicación. Asocia esta ruta con la función de vista Home del módulo views. Además, le asigna el nombre Ïnicio.ª esta ruta, lo que permite referenciarla fácilmente en otros lugares del código.
- Y de igual manera path('Bebidas/','Bocaditos/','Equipos/','Decoraciones/').Definen una ruta URL que corresponde a una parte de la página de decoraciones de la aplicación. Asocia esta ruta con la funcion correspondiente y le asigna el nombre "Decoraciones"..

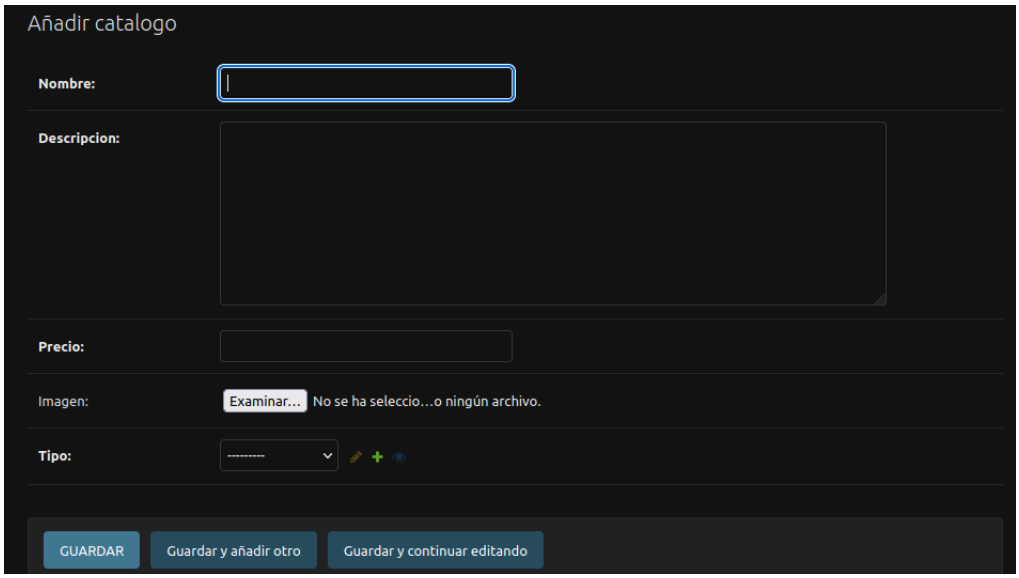
4.3. Ejecucion



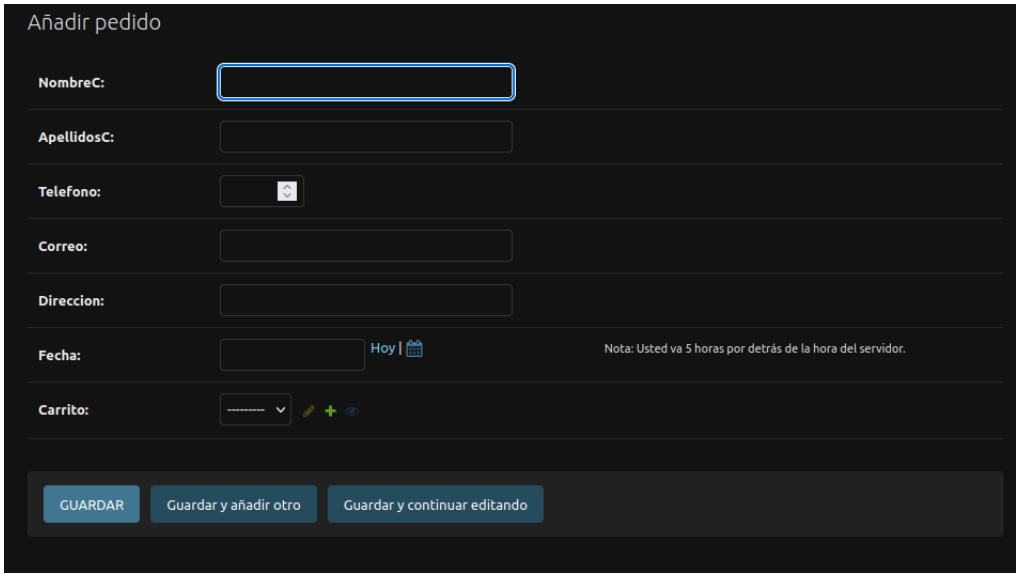
- En este caso se observa el administrador con todas las clases de models.py



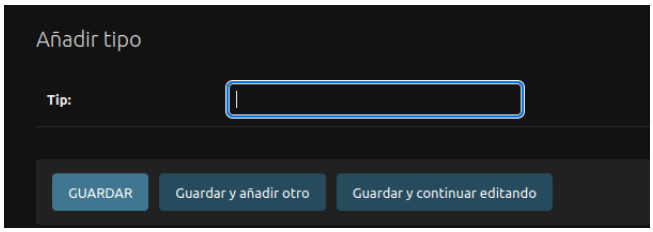
- En este caso se observa el administrador con la vista del formulacio de la clase Carritos de models.py



- En este caso se observa el administrador con la vista del formulacio de la clase Catalogos de models.py



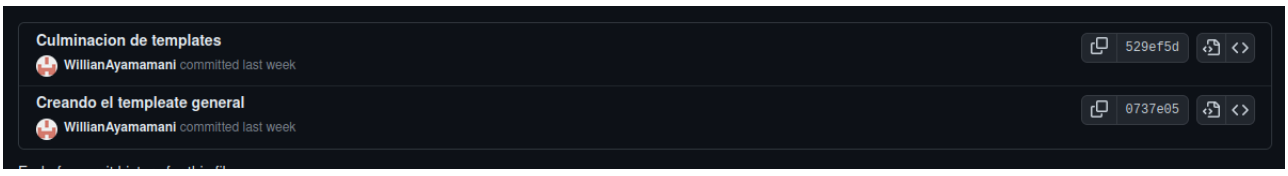
- En este caso se observa el administrador con la vista del formulacio de la clase Pedidos de models.py

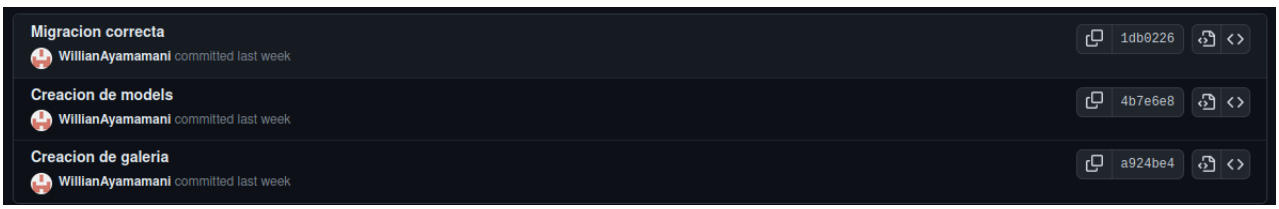


- En este caso se observa el administrador con la vista del formulacio de la clase Tipos de models.py

5. Commit-Estructura

5.1. Commit-Laboratorio 05





5.2. Estructura de laboratorio 05

- El contenido que se entrega en este laboratorio es el siguiente:

```
lab5/
|--- LATEX/
    |   |--- main.tex
    |   |--- lab5.pdf
    |   |--- img
            |   |--- .gitignore
            |   |--- Captura desde 2023-06-30 18-12-00.png
            |   |--- Captura desde 2023-06-30 18-12-00.png
            |   |--- Captura desde 2023-06-30 18-12-00.png
            |   |--- Captura desde 2023-06-30 18-12-00.png
            |   |--- Captura desde 2023-06-30 18-12-00.png
            |   |--- Ejercicio2e.py
            |   |--- Ejercicio2f.py
            |   |--- Ejercicio2g.py
            |   |--- chessPictures.py
            |   |--- colors.py
            |   |--- interpreter.py
            |   |--- picture.py
```

6. Cuestionario

- ¿Qué aprendiste?
  Aprendi a que mi trabajo de django se puede mejorar, con el uso de una App mas, asi tambien puedo utilizar el urls.py para vincular las rutas de mi sitio desde un punto principal.

7. Rúbricas

7.1. Entregable Informe

Tabla 1: Tipo de Informe

Informe	
Latex	El informe está en formato PDF desde Latex, con un formato limpio (buena presentación) y facil de leer.

7.2. Rúbrica para el contenido del Informe y demostración

- El alumno debe marcar o dejar en blanco en celdas de la columna **Checklist** si cumplio con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la siguiente tabla:

Tabla 2: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 3: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2	X	2	
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4	X	2	
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2	X	1	
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2	X	1	
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2	X	2	
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2	X	1	
7. Ortografía	El documento no muestra errores ortográficos.	2	X	2	
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4	X	1	
Total		20		12	

8. Referencias

- https://github.com/WillianAyamamani/pw2-lab-d-23a/tree/main/lab5